IT305 Socket Programming Project: Viva Complete Study Guide

Fault-Tolerant Topic-Based File Distribution Service

> **To All Group Members (PG6: Group 61 & Group 85):**
> The instructors announced: "*The evaluation will be conducted on a group basis. If even one member is unable to explain the project or answer questions, it may affect the entire group's marks.*"
>
> **Read this document carefully.** It explains the entire project implementation, execution flow, code functions, and gives you the exact answers to the top 25 questions the professors will ask.

Table of Contents

1. The 30-Second Elevator Pitch (Start Here)
2. End-to-End Execution Flow (What happens when you run the code?)
3. File-by-File Code Anatomy (Every struct, function, and file explained)
4. The 15 Must-Know System Calls & Sockets Concepts
5. The Top 25 Viva Questions & Model Answers
6. Quick Self-Test Checklist Before Entering the Viva Room

---

1. The 30-Second Elevator Pitch

If the examiner asks: **"Briefly tell me what you built in this project."**
Say this confidently:

> *"We built a fault-tolerant, high-performance file distribution service in C using raw POSIX TCP sockets to distribute multi-file image datasets (the Animals-10 dataset with 26,000+ files across 10 topics).
> The project evolves across four phases:
>
> 1. **Part 1:** Dynamic directory discovery and multi-threaded concurrent client serving.
> 2. **Part 2 Case 1:** Stateless fault recovery under active probabilistic connection drops (`SO_LINGER` TCP resets), where the client restarts from File 0.
> 3. **Part 2 Case 2:** Stateful resumption using packed binary headers, a server session table, client `.journal` bitmasks, and `lseek()`, reducing redundant retransmission from 104 MB to just 0.25 MB.
> 4. **Part 2 Case 2 Enhanced:** A parallel TCP architecture using 4 concurrent worker connections and a thread-safe work-stealing task queue, transferring 1,668 files (144 MB) in **1.92 seconds at ~630 Mbps** with independent per-stream fault recovery."*

---

2. End-to-End Execution Flow

Understanding the exact sequence of events is crucial. Walk through these steps:

A. When the Server Starts Up:

1. **Command Line Parsing:** Server reads `<port>`, `<topics_root_dir>`, and (in Part 2) `<failure_prob>`.
2. **Dynamic Directory Discovery (`topic_manager.c`):**
   - Calls `topic_registry_init()`.
   - Uses `opendir()` and `readdir()` on `../../raw-img` to find all subfolders (`gatto`, `cane`, etc.).
   - Uses `stat()` on each image file to record its exact filename, relative path, and byte size into a `TopicRecord`.
3. **Socket Setup (`common_net.c`):**
   - Calls `socket(AF_INET, SOCK_STREAM, 0)` to create an IPv4 TCP socket.
   - Sets `SO_REUSEADDR` so ports can be re-bound immediately without waiting for TIME_WAIT.
   - Binds to `INADDR_ANY` (port 8081–8084) and calls `listen(listen_fd, 128)`.
4. **Listening Loop:**
   - Server sits in `accept()`, waiting for client connections.
   - In multi-threaded mode, each `accept()` returns a new `client_fd`, and the server spawns a dedicated worker thread via `pthread_create()`.

B. When the Client Connects (Part 2 Case 2 Enhanced):

1. **Master Connection Initialization:**
   - Client connects to the server and sends `MSG_INIT_REQUEST` with the requested topic (e.g. `gatto`).
   - Server looks up `gatto`, generates a unique `session_id` (e.g. `2001`), creates an `ActiveSession` entry in RAM, and responds with `MSG_SESSION_MANIFEST` containing `total_files = 1668` and `total_bytes = 144.14 MB`.
2. **Task Queue Population (`task_queue.c`):**
   - Client initializes a thread-safe FIFO `TaskQueue` and pushes 1,668 `FileTask` items (one per image file).
3. **Worker Thread Launch:**
   - Client spawns $N = 4$ parallel worker threads (`parallel_worker_thread`).
4. **Worker Execution Loop (Work-Stealing):**
   - Each worker pops a task from the shared queue under `queue_lock`.
   - Worker opens its own TCP connection to the server.
   - Worker sends a 28-byte `MSG_TASK_REQUEST` with `FLAG_SINGLE_TASK`, `session_id`, `file_index`, and `start_offset`.
   - Server sends `MSG_RESUME_ACK`, then `MSG_FILE_START` (with filename), streams chunks via `MSG_DATA_PAYLOAD`, and ends with `MSG_FILE_COMPLETE`.
   - Worker writes bytes directly to its destination file in `/tmp/out_enhanced/gatto/`.
   - Worker updates the client's `.journal` checklist and grabs the next task from the queue!

C. What Happens During a Network Fault?

1. **Injected Failure:**
   - On the server, `fault_injector_should_fail()` randomly triggers based on failure probability p.

    - Server calls `execute_abrupt_disconnect()`: sets `SO_LINGER` (timeout 0) and closes the socket. This transmits an immediate **TCP RST** packet to the client.
2. **Client Recovery in Case 1 (Stateless):**
    - Client detects `recv() <= 0` (`ECONNRESET`).
    - Client completely deletes all downloaded files in `/tmp/out_case1/`.
    - Client reconnects and requests the topic again from **File 0, Byte 0**.
3. **Client Recovery in Case 2 (Stateful):**
    - Client detects disconnect at File 787, Byte 2529.
    - Client reconnects and sends `MSG_RESUME_REQUEST` with `file_index = 787, byte_offset = 2529`.
    - Server verifies the session table, seeks the file descriptor using `lseek(file_fd, 2529, SEEK_SET)`, and resumes streaming right from byte 2529. Completed files (0 to 786) are skipped!
4. **Client Recovery in Case 2 Enhanced (Parallel TCP):**
    - If Worker 3's connection drops, Worker 3 updates `task.start_offset = current_offset`, pushes the interrupted task back onto the `TaskQueue`, and reconnects.
    - **Workers 1, 2, and 4 keep downloading their files completely uninterrupted!**

---

3. File-by-File Code Anatomy

Every member should know what each file in `FinalSubmission_Group_061_085/` does:

1. `protocol.h` (The Framing Contract)

- **Purpose:** Defines the fixed 28-byte binary header used in Case 2 and Enhanced.
- **Key Struct:**

```
#pragma pack(push, 1) // Tells compiler NOT to add padding bytes!
typedef struct {
    uint32_t magic;         // PROTOCOL_MAGIC = 0x434E534F ('CNSO')
    uint8_t  msg_type;      // Opcode (MSG_INIT_REQUEST, MSG_DATA_PAYLOAD,
etc.)
    uint8_t  flags;         // Control flags (FLAG_SINGLE_TASK)
    uint16_t status_code;   // STATUS_OK (0), STATUS_UNKNOWN_TOPIC (404),
etc.
    uint32_t session_id;    // 32-bit unique session identifier
    uint32_t file_index;    // Which file in topic (0 to N-1)
    uint64_t byte_offset;   // Byte offset inside file (for lseek)
    uint64_t payload_len;   // Byte length of data following this header
} ProtocolHeader;
#pragma pack(pop)
```

- **Key Functions:** `header_to_net()` and `header_from_net()` convert fields between Host and Network byte order using `htonl()`, `ntohl()`, `htobe64()`, and `be64toh()`.

---

2. `common_net.c` & `common_net.h` (Network Primitives)

- **Purpose:** Core socket communication functions used by both client and server.

- **Key Functions:**
  - `send_exact(int fd, const void *buf, size_t count)`: Loops `send()` until all `count` bytes are transmitted.
  - `recv_exact(int fd, void *buf, size_t count)`: Loops `recv()` until all `count` bytes are received. Prevents partial reads and buffer bleed!
  - `create_server_socket(int port)`: Creates, sets `SO_REUSEADDR`, binds, and listens on port.
  - `connect_to_server(const char *ip, int port)`: Resolves address via `getaddrinfo()`, connects, and sets `TCP_NODELAY`.

---

3. `topic_manager.c` & `topic_manager.h` (Dataset Discovery)

- **Purpose:** Scans the dataset directory without hardcoded names.
- **Key Structs:** `TopicRegistry`, `TopicRecord`, `FileRecord`.
- **Key Functions:**
  - `topic_registry_init(reg, root_dir)`: Opens `root_dir` with `opendir()`, reads each folder with `readdir()`, stats every image file (`stat()`), and stores file size and path.
  - `topic_registry_find(reg, name)`: Fast topic lookup.
  - Uses `pthread_rwlock_t` to allow multiple threads to read topic metadata concurrently with zero locking contention.

---

4. `fault_injector.c` & `fault_injector.h` (Active Fault Engine)

- **Purpose:** Simulates network crashes deterministically or probabilistically.
- **Key Functions:**
  - `fault_injector_should_fail(p_failure, filesize, ...)`: Performs a Bernoulli trial using `rand_r()`. If random value $\leq p$, picks a random byte offset between 0 and `filesize`.
  - `execute_abrupt_disconnect(sock_fd, ...)`: Configures `SO_LINGER` with timeout 0 and calls `close()`, transmitting a TCP RST flag.

---

5. `session_manager.c` & `session_manager.h` (Server Session Memory)

- **Purpose:** Tracks client sessions in RAM for stateful resumption (Case 2 & Enhanced).
- **Key Structs:**
  - `ActiveSession`: Stores `session_id`, `topic`, `total_files`, array of `FileProgress`, and `last_active` timestamp.
  - `SessionRegistry`: Table of up to 256 active sessions, protected by `registry_lock`.
- **Key Functions:**
  - `session_create()`: Allocates a new session and returns a unique `session_id`.
  - `session_find()`: Looks up session by ID upon client reconnection.
  - `session_update_progress()`: Updates the last confirmed byte offset and completion status.
  - `SESSION_TTL_SEC` (300s): Automatically evicts inactive sessions after 5 minutes.

---

6. `task_queue.c` & `task_queue.h` (Work-Stealing Queue in Enhanced)

- **Purpose:** Distributes 1,668 file download tasks across 4 parallel TCP worker threads.

- **Key Struct:**
  - `TaskQueue`: Circular FIFO queue protected by `pthread_mutex_t queue_lock` and `pthread_cond_t queue_cond`.
- **Key Functions:**
  - `task_queue_push()`: Pushes a task and signals waiting workers via `pthread_cond_signal()`.
  - `task_queue_pop()`: Workers safely retrieve the next available file task.
  - `task_queue_is_finished()`: Checks if all 1,668 files have been completed.

---

4. The 15 Must-Know System Calls & Sockets Concepts

Examiners frequently point to a line of C code and ask: *"What does this system call do?"*
Here is the cheat-sheet:

System Call / Concept	Exact Purpose in Our Project
`socket()`	Creates the endpoint for communication; returns a file descriptor.
`bind()`	Binds the socket to an IP (`INADDR_ANY`) and port number (e.g., 8081).
`listen()`	Marks the socket as passive, ready to accept incoming connections (backlog queue: 128).
`accept()`	Blocks until a client connects; returns a brand new socket file descriptor for that client.
`connect()`	Used by the client to initiate a 3-way TCP handshake with the server.
`send()` & `recv()`	Transmit and receive raw bytes over a connected stream socket.
`lseek()`	**The heart of Case 2:** Repositions the file read offset to resume directly from byte X.
`SO_REUSEADDR`	Socket option allowing instant re-binding to a port without waiting for `TIME_WAIT`.
`SO_LINGER`	Socket option configured with linger time 0 to send a **TCP RST** (hard crash simulation).
`TCP_NODELAY`	Disables Nagle's algorithm; sends small protocol headers immediately with 0 ms delay.
`opendir()` / `readdir()`	Traverses directories to dynamically discover topics and files without hardcoding.
`stat()`	Retrieves file metadata (specifically `st_size` for exact file length in bytes).
`pthread_create()`	Spawns a new concurrent thread (used for multi-client serving and 4 parallel workers).
`pthread_mutex_t`	Mutual exclusion lock protecting shared memory (`SessionRegistry`, `TaskQueue`).

System Call / Concept	Exact Purpose in Our Project
`pthread_cond_t`	Condition variable used to put worker threads to sleep until new tasks are queued.

5. The Top 25 Viva Questions & Model Answers

Category A: Architecture & Networking Basics

Q1: Why did we choose TCP instead of UDP for this file distribution service?

> **Answer:** TCP provides reliable, in-order, error-checked delivery of byte streams through sequence numbers and acknowledgments. Since we are distributing image files, missing or out-of-order packets would corrupt images. UDP has no delivery guarantees, making TCP the appropriate transport protocol.

Q2: What is "Buffer Bleed" and how does your project solve it?

> **Answer:** TCP is a continuous byte stream without message boundaries. If you send a filename followed immediately by file bytes, TCP can merge them into a single packet, or split them across multiple packets. A receiver calling `read()` might get half the filename and some file bytes merged together (Buffer Bleed). We solve this by implementing `send_exact()` and `recv_exact()` and using fixed-size binary headers (`ProtocolHeader`) where the exact byte length of the payload is stated upfront.

Q3: What is the purpose of `#pragma pack(push, 1)` in `protocol.h`?

> **Answer:** In C, compilers automatically insert padding bytes between struct members to align them to 32-bit or 64-bit memory boundaries. If the server and client compilers use different padding, the struct bytes over the wire won't match. `#pragma pack(push, 1)` forces the compiler to pack the struct with 1-byte alignment (zero padding), guaranteeing that `sizeof(ProtocolHeader)` is exactly 28 bytes on any machine.

Q4: Why do we call `htons()`, `htonl()`, and `htobe64()`?

> **Answer:** To handle Endianness (Byte Ordering). x86 processors store multi-byte integers in Little-Endian format (least significant byte first), whereas Internet protocols require Big-Endian format (Network Byte Order). These functions convert host integers to network byte order before sending, and `ntohl()`/`be64toh()` convert them back on arrival.

Category B: Concurrency & Multi-Threading

Q5: What is the difference between the Single-Threaded and Multi-Threaded server in Part 1?

> **Answer:** The Single-Threaded server handles clients sequentially. If Client 1 is downloading a 150 MB topic, Client 2 is blocked in the backlog queue until Client 1 finishes. The Multi-Threaded server calls `pthread_create()` upon each `accept()`, spawning a dedicated thread for each client so multiple clients download simultaneously without blocking.

Q6: How do you prevent Race Conditions when multiple threads access the same data?

> **Answer:** We protect shared resources using POSIX Mutexes (`pthread_mutex_t`). For example, in `session_manager.c`, the `registry_lock` protects the active sessions array during registration and lookups. In `task_queue.c`, `queue_lock` ensures only one worker thread can pop a task at a time. For topic metadata, we use a read-write lock (`pthread_rwlock_t`), which allows multiple threads to read topic catalogs concurrently without blocking.

Q7: In Enhanced Case 2, why did you use a dynamic Task Queue instead of statically dividing files (e.g. 400 files per worker)?

> **Answer:** Dynamic work-stealing provides automatic load balancing. In the Animals-10 dataset, image file sizes vary significantly (from 10 KB to 250 KB). If we statically assigned 400 files to each worker, a worker given large files would take much longer, leaving other workers idle. With our dynamic FIFO queue, whenever a worker finishes a file, it immediately pops the next task, ensuring all 4 connections remain 100% utilized.

Category C: Fault Tolerance & Recovery

Q8: How does your server simulate an unexpected network failure?

> **Answer:** In `fault_injector.c`, we configure socket option `SO_LINGER` with `l_onoff = 1` and `l_linger = 0`. When `close()` is called on that socket, the OS kernel bypasses the normal graceful 4-way TCP `FIN` handshake and immediately transmits a **TCP RST (Reset)** packet. This instantly severs the connection on both ends, simulating an abrupt network failure or cable cut.

Q9: What is the difference between Stateless Recovery (Case 1) and Stateful Recovery (Case 2)?

> **Answer:** In Stateless Recovery (Case 1), neither client nor server stores checkpoint progress. When a connection drops, the client wipes all downloaded files and restarts the entire transfer from File 0, Byte 0. In Stateful Recovery (Case 2), the client records completed files in a disk `.journal` bitmask, and the server tracks session state in RAM. On reconnect, the client requests resumption from its exact file and byte offset, and the server seeks with `lseek()`.

Q10: Why did Case 1 waste over 104 MB of data on topic `gatto`?

> **Answer:** Under a 20% failure probability, the connection was aborted 11 times. Each time it failed on file 400, 800, or 1200, the client threw away all previously downloaded files and re-downloaded them from the start. Over 11 restarts, 104 MB of duplicate image bytes were repeatedly sent across the wire.

Q11: How does the server avoid reading from the start of a file when resuming in Case 2?

> **Answer:** The client sends the interrupted `byte_offset` in its `MSG_RESUME_REQUEST`. The server opens the file and calls `lseek(file_fd, (off_t)byte_offset, SEEK_SET)`. This system call repositions the kernel file offset pointer directly to that byte, skipping all previously delivered bytes with zero disk or network re-reading.

Q12: What happens if a client crashes and never reconnects in Case 2? Does the server leak memory?

> **Answer:** No. In `session_manager.c`, every session has a `last_active` timestamp. We enforce a Session TTL of 300 seconds (`SESSION_TTL_SEC = 300`). Inactive sessions that exceed 5 minutes without activity are automatically reclaimed and evicted from the `ActiveSession` table.

Category D: Performance & The 20-Second Optimization

Q13: What is "Protocol Overhead" and what was the value in your tests?

> **Answer:** Protocol Overhead is the percentage of total transmitted network bytes wasted on non-payload data:
>
> $$\text{Overhead } \% = \frac{\text{Total Transmitted Bytes} - \text{Useful Bytes}}{\text{Useful Bytes}} \times 100\%$$
>
> In Case 1, overhead was **~72%** due to 104 MB of redundant retransmissions. In Case 2 and Enhanced, overhead dropped to **~0.17% – 0.24%** (only 0.25 MB overhead on a 144 MB transfer), achieving **99.8% useful data efficiency**.

Q14: What is Effective Throughput (Goodput) vs. Raw Throughput?

> **Answer:** Raw throughput measures total bytes passing through the socket per second, including wasted retransmissions and headers. Effective Throughput (Goodput) measures only the **unique, useful payload bytes** delivered to disk per second. In Case 1, raw throughput seemed normal, but effective throughput was low (~32 Mbps) because time was wasted sending duplicate files. In Enhanced Case 2, effective throughput reached **~630 Mbps**.

Q15: Why did Parallel TCP initially take 20.8 seconds, and how did you reduce it to 1.92 seconds?

> **Answer:** Initially, all 4 worker threads competed for a single mutex lock to execute a synchronous hardware disk flush (`fsync`) on the `.journal` file after every single 50 KB image. Doing 1,668 hardware `fsync` calls serialized the threads and wasted ~17 seconds in disk I/O. We applied two optimizations:
>
> 1. Removed `fsync` from the per-file loop, allowing updates to be buffered in the OS RAM page cache (< 0.001 ms).
> 2. Enabled `TCP_NODELAY` on all sockets to disable Nagle's algorithm and eliminate 40 ms delayed-ACK pauses.
>
> This dropped elapsed time from **20.8s down to 1.92s**, giving a **~10x speedup**.

Q16: What is Nagle's Algorithm and why did you disable it (`TCP_NODELAY`)?

> **Answer:** Nagle's algorithm buffers small outbound TCP packets to save bandwidth, waiting for an acknowledgment before sending another small packet. Since our workers frequently exchange small 28-byte headers (`MSG_TASK_REQUEST` and `MSG_RESUME_ACK`), Nagle caused artificial 40 ms delayed-ACK pauses. Setting `setsockopt(..., TCP_NODELAY)` forces the socket to transmit small headers immediately without buffering.

Q17: What is the "Small File Problem" in computer networks?

> **Answer:** When transferring thousands of tiny files (e.g. 50 KB images), the cumulative overhead of per-file operations (creating sockets, task request round-trips, `open()`, `close()`) can dominate transfer time compared to actual data streaming. Over local loopback, a single continuous pipeline can sometimes match multi-stream setups due to request overhead. Parallel TCP truly excels when files are larger (MBs to GBs) or when network RTT is non-negligible (WAN/Internet).

Category E: Verification & Real-World Scaling

Q18: How do you prove that the transferred files are not corrupted?

> **Answer:** We run the Linux command:
>
> ```
> diff -r ../../raw-img/gatto /tmp/out_enhanced/gatto
> echo $?
> ```
>
> `diff -r` compares every file recursively byte-for-byte. If all 1,668 files match with zero differences, `diff` outputs nothing and the shell returns exit code `0`. We verified this across all test runs.

Q19: What would you do if the topic contained one giant 4 GB video file instead of 1,668 small files?

> **Answer:** Our system already supports byte-offset resumption for large files. To achieve parallel multi-stream speedup on a single giant file, we would implement **Byte-Range Slicing (Chunking)** (similar to BitTorrent or IDM). Worker 1 downloads bytes 0–1GB, Worker 2 downloads 1–2GB, Worker 3 downloads 2–3GB, and Worker 4 downloads 3–4GB. All 4 workers write simultaneously into the same destination file using `pwrite(fd, buf, count, offset)`.

Q20: Why does your server bind to `INADDR_ANY`?

> **Answer:** `INADDR_ANY` (0.0.0.0) binds the listening socket to all available network interfaces on the host (e.g., Ethernet, Wi-Fi, and loopback). This allows the server to accept connections not just from `127.0.0.1` locally, but also from other physical laptops over LAN or Wi-Fi (satisfying the 3-PC lab demonstration requirement).

Q21: What is the purpose of the Client Journal Bitmask (`completed_mask[]`)?

> **Answer:** It is a memory-efficient checklist where each bit represents the completion status of one file. Since 1 byte has 8 bits, an array of 2,048 bytes tracks up to 16,384 files. When File i is complete, the client sets `completed_mask[i / 8] |= (1 << (i % 8))`. On resumption, the client checks this bit in $O(1)$ time to skip already-downloaded files.

Q22: How does the server prevent Directory Traversal security attacks?

> **Answer:** In `server.c`, before looking up any requested topic name, the server sanitizes the query:
>
> ```
> if (strstr(topic, "..") != NULL || strchr(topic, '/') != NULL ||
> strchr(topic, '\\') != NULL) {
> ```

```
        // Send ERROR SECURITY_VIOLATION and drop client
    }
```

This prevents malicious clients from escaping the dataset folder by sending `GET ../../../etc/passwd`.

Q23: Why did we use `rand_r()` instead of standard `rand()` in server threads?

> **Answer:** Standard `rand()` is not thread-safe; it uses a shared global internal seed, which causes race conditions and lock contention when multiple worker threads call it concurrently. `rand_r()` takes a pointer to an independent per-thread seed (`unsigned int *seed`), making it re-entrant and completely thread-safe.

Q24: What caused the server thread seed collision bug and how did you resolve it?

> **Answer:** Initially, worker threads seeded their PRNG with `time(NULL) ^ pthread_self()`. In Linux `glibc`, short-lived worker threads reuse the same thread control block address from the thread cache within the same second, resulting in identical random seeds. This caused repetitive back-to-back aborts. We resolved it by seeding with `clock_gettime(CLOCK_MONOTONIC)` nanoseconds mixed with an atomic sequence counter (`__sync_fetch_and_add`), guaranteeing unique seeds across all reconnections.

Q25: Why did we compile with `-Wall -Wextra -Werror -O2 -pthread`?

> **Answer:**
>
> - `-Wall` and `-Wextra`: Enable all major and extra compiler warning diagnostics.
> - `-Werror`: Treats all warnings as errors, ensuring production-grade code cleanliness.
> - `-O2`: Enables level-2 compiler optimization (loop unrolling, register allocation, instruction scheduling).
> - `-pthread`: Links the POSIX threads library and sets re-entrant preprocessor flags.

---

6. Quick Self-Test Checklist

Before you walk into the lab room tomorrow, ensure every team member can answer these 4 questions without looking at notes:

1. *What command runs the Enhanced server?* (`./server 8084 ../../raw-img 0.50`)
2. *What command runs the Enhanced client with 4 streams?* (`./client <IP> 8084 gatto /tmp/out 4`)
3. *What does `lseek()` do in Case 2?* (Jumps directly to the interrupted byte offset without re-reading previous bytes).
4. *How did we get from 20 seconds to 1.92 seconds?* (Removed per-file hardware `fsync` lock contention + enabled `TCP_NODELAY`).

Good luck to Group 61 & Group 85! You are 100% prepared to ace the viva!